# VEHICLE RENTAL MANAGEMENT SYSTEM

Design Report | Object-Oriented Programming

## 1. PROJECT OVERVIEW

The Vehicle Rental Management System is a Java console application for a small rental business. It manages cars, motorcycles, trucks, customer records, rentals, returns, and maintenance. The menu supports adding and finding vehicles, registering customers, renting and returning vehicles, viewing rental histories, calculating completed-rental revenue, and maintaining the fleet.

The application separates the console interface from business rules. Main presents the menu, reads user input, and displays results. RentalSystem coordinates operations and manages the vehicle, customer, and rental collections. Vehicle, Customer, and Rental represent the main domain concepts. This division keeps prompts and display logic separate from the rules that govern rentals.

2. ABSTRACTION AND THE CLASS HIERARCHY

Vehicle is an abstract base class because the program rents concrete categories rather than a generic vehicle. It encapsulates the shared ID, brand, model, manufacturing year, daily rental price, availability, and maintenance state. It also supplies common operations for updating price, checking availability, flagging maintenance, and completing a rental.

Car, Motorcycle, and Truck extend Vehicle and add their specific attributes: a car's seat count and transmission type, a motorcycle's engine capacity, and a truck's maximum load. Constructors use super to initialize shared vehicle data before subtype fields. Overloaded constructors offer supported ways to create objects. The keyword this distinguishes object fields from constructor or method parameters with the same name.

## 3. ABSTRACT CLASS AND INTERFACE

An abstract class provides related types with common state and implementation while requiring concrete classes to supply specialized behavior. Vehicle fulfills that role: it stores shared vehicle information, but concrete subclasses define their own rental calculation and descriptive details. A generic Vehicle is not complete enough to instantiate.

# 4. POLYMORPHISM, OVERRIDING, AND OVERLOADING

RentalSystem stores fleet members through Vehicle references. When it requests a cost or vehicle description, Java dispatches the call to the concrete object's implementation at runtime. This is polymorphism: the same operation behaves appropriately for each vehicle category without RentalSystem branching on vehicle type.

A car charges its daily rate multiplied by the rental days, with optional insurance at $15 per day. A motorcycle applies the daily calculation and can add a helmet at $5 per day. A truck adds a fixed $50 service fee per rental. Each subtype owns its cost rule, optional-service prompt, and service description. Adding another kind of vehicle can add its pricing behavior in its own class while reusing the rental workflow.

Overriding lets a subtype provide a specialized implementation of an inherited or abstract method. Overloading uses one method name with different parameter lists. For example, RentalSystem supports searching by vehicle ID or by text query, and cost calculation with or without an optional-service selection. Overloading is resolved from the arguments at compile time; overriding is chosen from the object's runtime class.

The code uses static constants for shared pricing amounts and a static vehicle counter for class-wide information. Final declarations protect values and operations that should not be reassigned or overridden. These keywords make sharing and design constraints explicit.

# 5. RENTALS, CUSTOMERS, AND STATE

Customer stores a unique ID, name, phone, email, and driving-license number. Contact details can be updated. Rental links a customer and a vehicle and records duration, total cost, optional service, start date, return date, and active or completed status. Rental history is derived from the associated records.

RentalSystem checks rules before changing state. It rejects duplicate identifiers, missing records, nonpositive rental durations, unavailable vehicles, and vehicles flagged for maintenance. A vehicle may have only one active rental, while a customer may rent several vehicles. Returning an active rental completes it and releases the vehicle; returning it again is rejected. Revenue sums completed rentals only, excluding active transactions.

# 6. COLLECTIONS AND VALIDATION

RentalSystem manages the changing fleet, customer register, and rental history with ArrayList collections. Accessors provide snapshots so callers can inspect records without editing the internal lists. Input and domain checks reject invalid values and state transitions with clear exceptions. Main catches expected errors and reports them while keeping the menu available.

## 7. MAINTENANCE WORKFLOW

A vehicle can be flagged for maintenance even while rented. The flag prevents new rentals once it becomes available. Maintenance cannot be performed while the vehicle has an active rental. After the vehicle is returned, maintenance can be completed; the operation records the work and clears the flag. The system lists vehicles needing maintenance and invokes behavior through Maintainable references so that the workflow applies across vehicle classes.

## 8. MENU AND CLASS RESPONSIBILITIES

The menu covers adding, searching, and displaying vehicles; registering and listing customers; renting and returning vehicles; viewing all rentals and customer history; calculating completed revenue; and flagging, listing, and performing maintenance. Additional actions update vehicle prices and customer phone or email details and display a customer's active rentals.

Main owns prompts, parsing, and presentation. RentalSystem owns coordination and business constraints. Customer and Rental own their respective records. Vehicle manages shared rental and maintenance state; subclasses supply vehicle-specific pricing and descriptions. This separation gives each class a clear responsibility and keeps business calculations outside the console menu.

## 9. CONCLUSION

The design uses encapsulation to protect object state, inheritance to reuse common vehicle data and behavior, and abstraction to define the shared Vehicle contract. Maintainable expresses a capability contract. Runtime polymorphism lets the service layer calculate costs and perform maintenance through common references. Overloading provides related operations with different parameters; overriding supplies specialized subtype behavior.

These choices keep pricing rules with the vehicle classes and rental workflow in RentalSystem. A new vehicle subtype can participate by adding its attributes and required behavior while reusing the customer, rental, and menu flow. Together, the classes demonstrate object-oriented design in an extendable console application.

## OOP CONCEPTS DEMONSTRATED

Encapsulation — private state with validated methods.
Inheritance — Car, Motorcycle, and Truck extend Vehicle.
Abstraction — Vehicle defines common state and required vehicle behavior.
Interface — Maintainable expresses the maintenance capability.
Polymorphism — common references dispatch to vehicle-specific implementations.
Overloading and overriding — related operations and specialized subtype behavior.